

# 将 VisEval 数据集转换为 BA-bench 格式

## 目录

1. 引言
  2. BA-bench 数据格式规范
  3. VisEval 数据集详解
    - 目标与评估维度
    - 核心数据结构
  4. 转换策略：VisEval 到 BA-bench
    - 核心思想
    - 字段映射详解
  5. 代码实现详解 (`TransVis/transvis.py`)
    - 依赖库
    - 函数: `extract_all_table_names(sql_query)`
    - 函数: `convert_viseval_to_ba_bench(...)`
    - 脚本执行与路径配置
  6. 数据格式建议修改
- 

## 1. 引言

本文档旨在阐述将 VisEval 数据集转换为 BA-bench (Business Analytics Benchmark for GenAI Agents) 标准格式的过程。首先将分别介绍 BA-bench 的目标格式和 VisEval 数据集的结构与特点，然后说明二者之间的转换逻辑与策略，最后解析实现该转换的 Python 脚本 (`TransVis/transvis.py`) 的具体代码

## 2. BA-bench 数据格式规范

根据项目 `README.md` 文件定义，BA-bench 旨在为评估生成式 AI Agent 在商业分析任务上的能力提供一个统一的基准。每个数据样本（代表一个具体的问题）都应遵循以下 JSON 结构：

```
{
    'id': str, # 每个样本的唯一标识符
    'question': str, # 与数据相关的业务问题或指令
    'data_file': str, # 问题相关的 *主要* 数据文件名（相对路径）
    'doc_file': str, # 问题相关的文档文件名（若无则为 'None'）
    'answer': str, # 问题的答案（字符串形式）
    'data_domain': str, # 数据所属的领域（例如：金融，教育，可视化）
    'analysis_type': str, # 问题类型，例如：["Structure problems",
    "Unstructured problems", "Chart problems"]
    'origin_from': list[str], # 问题的来源，例如：['benchmark
    name', 'original id']
    'additional_information': dict, # 包含其他附加信息的字典，结构灵
    活
}
```

### 关键字段说明：

- **id**: 唯一标识符，方便追踪和引用样本。
- **question**: Agent 需要理解和回答的自然语言问题或指令。
- **data\_file**: 指向解决问题所需的主要数据文件的路径（字符串）。**注意**：此字段设计为单个字符串。
- **doc\_file**: 指向可能需要的辅助文档。
- **answer**: 问题的“标准答案”。根据 **analysis\_type** 不同，其内容可能是一个数值、一段文本、或者一个序列化的对象（如 JSON 字符串）。**关键是类型必须是 str**。
- **data\_domain**: 问题和数据所属的领域分类。
- **analysis\_type**: 区分不同类型的问题，如涉及结构化答案、非结构化答案或图表生成。
- **origin\_from**: 记录样本的原始来源，便于追溯。
- **additional\_information**: 一个字典，用于存储未被顶级字段覆盖但对理解或评估样本有用的附加信息，例如原始 SQL 查询、难度级别、涉及的所有数据文件列表等。

## 3. VisEval 数据集详解

VisEval 是一个专门为评估（自然语言到可视化，NL2VIS）方法而设计的基准测试

### 3.1. 目标与评估维度

根据其 **README.md**，VisEval 主要从三个维度评估 Agent 生成可视化的能力：

1. **代码有效性**: Agent 生成的代码是否能够无误执行并产生输出。
2. **可视化正确性**: 生成的可视化是否准确地反映了用户查询（自然语言指令）的意图。
3. **可视化可读性**: 生成的可视化是否清晰、易于理解。

VisEval 数据集本身提供了执行这些评估所需的 **输入和参考标准**。

## 3.2. 核心数据结构

根据 `docs/data.md` 和对数据集文件的观察，VisEval 的核心数据 (`visEval_dataset.zip` 解压后) 包含：

1. `visEval.json` (以及 `visEval_single.json`, `visEval_multiple.json`):
  - 这是主要的元数据文件，格式为 JSON 对象。
  - 顶级键是原始样本 ID (字符串，如 `"8"`, `"9"`)。
  - 每个样本值是一个字典，包含以下关键信息：
    - `db_id` (str): 关联的数据库名称 (对应 `databases/` 下的子目录)。
    - `nl_queries` (list[str]): 一个或多个自然语言查询，都指向同一个目标可视化。这是 Agent 的主要输入提示。
    - `vis_query` (dict): 包含结构化的查询信息，特别是 `data_part['sql_part']` (str)，即用于从数据库提取数据的 SQL 查询。
    - `vis_obj` (dict): 核心部分，一个描述目标可视化规范的 JSON 对象。包含图表类型 (`chart`)、轴信息 (`x_name`, `y_name`)、用于绘图的数据 (`x_data`, `y_data`)、分组/排序要求等。这可以被视为可视化的“标准答案”。
    - `chart` (str): 明确的图表类型 (如 `"Pie"`, `"Bar"`)。
    - `hardness` (str): 难度级别。
    - 其他元数据，如 `irrelevant_tables`, `query_meta`。
2. `databases/` (目录):
  - 包含多个子目录，每个子目录名对应一个 `db_id`。
  - 每个 `db_id` 子目录下存放着该数据库对应的 **CSV 文件**，每个 CSV 文件代表一个数据表。这些是 SQL 查询实际操作的数据源。

## 4. 转换策略：VisEval 到 BA-bench

将 VisEval 数据转换为 BA-bench 格式需要仔细映射字段，并处理 VisEval 数据集的一些特性 (如多 `nl_queries` 和多文件依赖)

### 4.1. 核心理念

1. 每个 NL Query 一个样本: VisEval 的一个条目可能包含多个 `nl_queries`，但它们都指向同一个 `vis_obj`。为了让 BA-bench 中的每个样本对应一个明确的问题，将为原始数据中的每一个 `nl_query` 创建一个独立的 BA-bench 样本
  - 注意，如果之后 `question` 字段修改为列表，此处可以使用列表表达针对同一解答的多个问题
2. 保留目标规范: VisEval 的 `vis_obj` 是评估可视化正确性的关键，将其完整包含在 BA-bench 样本中
3. 处理多文件依赖: 对于需要 `JOIN` 多个表的查询，需要记录所有相关的数据文件，而不仅仅是一个
4. 遵循 BA-bench Schema: 转换结果必须严格遵守 BA-bench 的字段类型定义 (如 `answer: str`, `data_file: str`)

## 4.2. 字段映射详解

| BA-bench 字段                         | 类型                     | 来源/转换逻辑                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                     | <code>str</code>       | 生成唯一 ID，格式: <code>f"VisEval_{original_</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>question</code>               | <code>str</code>       | 来源于当前处理的 <code>vis_entry['nl_queries'</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>data_file</code>              | <code>str</code>       | 提取 <code>vis_entry['vis_query']['sql_part'</code><br>( <code>primary_table_name</code> ), 结合 <code>vis_entry['</code><br><code>VisEval/visEval_dataset/databases/{db</code>                                                                                                                                                                                                                                                                                                                                                                |
| <code>doc_file</code>               | <code>str</code>       | 固定设置为 <code>"None"</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>answer</code>                 | <code>str</code>       | 将 <code>vis_entry['vis_obj']</code> 字典通过 <code>json.d</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>data_domain</code>            | <code>str</code>       | 固定设置为 <code>"General Visualization"</code> (或可                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <code>analysis_type</code>          | <code>str</code>       | 固定设置为 <code>"Chart problems"</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <code>origin_from</code>            | <code>list[str]</code> | 固定设置为 <code>['VisEval', original_id]</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <code>additional_information</code> | <code>dict</code>      | 包含以下键值对: <ul style="list-style-type: none"><li>- <code>hardness</code>: <code>vis_entry['hardness']</code></li><li>- <code>chart</code>: <code>vis_entry['chart']</code></li><li>- <code>vis_query</code>: <code>vis_entry['vis_query']</code> (完</li><li>- <code>sql_part</code>: 原始 SQL 字符串</li><li>- <code>involved_data_files</code>: <code>list[str]</code>, 包含<br/>件路径列表。</li><li>- <code>irrelevant_tables</code>: <code>vis_entry['irrele</code></li><li>- <code>query_meta</code>: <code>vis_entry['query_meta']</code></li></ul> |

## 5. 代码实现详解 (TransVis/transvis.py)

以下是对 `TransVis/transvis.py` 脚本关键部分的详细解释:

## 5.1. 函数: `extract_all_table_names(sql_query)`

PYTHON

```
def extract_all_table_names(sql_query):
    """
    从 SQL 查询中提取 FROM 和 JOIN 子句后的所有表名。
    尝试处理简单的 AS 别名。返回一个唯一的表名列表。
    """
    # 正则表达式解释:
    # (?:(?:FROM|JOIN)) : 非捕获组, 匹配 "FROM" 或 "JOIN" 关键字。
    # \s+                : 匹配关键字后的一个或多个空白字符。
    # ([\w.-]+)          : 捕获组 (这是我们想要的表名)。
    #                    [\w.-]+ 匹配一个或多个字母、数字、下划线、点或连字符。
    # (?:(?:\s+AS\s+\w+)?) : 非捕获组, 可选地匹配 " AS alias" (处理别名, 但不捕获它)。
    # re.IGNORECASE      : 忽略 SQL 关键字和表名的大小写。
    tables = re.findall(r'(?:(?:FROM|JOIN))\s+([\w.-]+)(?:(?:\s+AS\s+\w+)?)', sql_query, re.IGNORECASE)

    # 使用 dict.fromkeys 去重, 同时保持找到的顺序
    unique_tables = list(dict.fromkeys(tables))

    # 后备逻辑: 如果 findall 没找到 (可能 SQL 结构复杂)
    if not unique_tables:
        # 尝试只从 FROM 后面匹配第一个表名
        fallback_match = re.search(r'FROM\s+([\w.-]+)', sql_query, re.IGNORECASE)
        if fallback_match:
            unique_tables = [fallback_match.group(1)] # 如果找到, 返回包含这一个表名的列表
        else:
            # 如果连 FROM 都匹配不到, 打印警告
            print(f"警告: 无法从 SQL 中提取任何表名: {sql_query}")
            # 返回空列表 (调用者需要处理这种情况)
    return unique_tables
```

- **目的:** 从给定的 SQL 查询字符串中提取所有引用的数据表名称
- **方法:** 主要使用 `re.findall` 和一个正则表达式来查找 `FROM` 或 `JOIN` 关键字后面的表名。该正则表达式设计得可以处理大小写和简单的 `AS` 别名
- **去重与保序:** 利用 `dict.fromkeys()` 方法可以有效地去除重复的表名, 同时保持它们在 SQL 查询中出现的相对顺序

- **后备机制:** 如果主要的正则表达式没有找到任何匹配（例如，非常规的 SQL 结构），它会尝试一个更简单的模式，只查找 **FROM** 后面的第一个词作为表名。如果连这个也失败了，就打印警告并返回空列表

## 5.2. 函数: `convert_viseval_to_ba_bench(...)`

PYTHON

```
def convert_viseval_to_ba_bench(input_json_path, output_json_path,
                                base_data_dir):
    # ... (错误处理: 打开输入文件) ...

    ba_bench_data = [] # 初始化列表以存储所有转换后的样本
    processed_count = 0 # 计数器: 成功处理的 nl_query 数量
    error_count = 0     # 计数器: 跳过的原始条目/查询数量

    # 1. 遍历 VisEval JSON 数据中的每个原始条目 (ID 和内容)
    for original_id, vis_entry in viseval_data.items():
        # 2. 获取 nl_queries 列表, 如果为空则跳过
        nl_queries = vis_entry.get('nl_queries', [])
        if not nl_queries:
            error_count += 1
            continue

        # 3. 获取关键信息 (db_id, sql_part, vis_obj), 如果缺少则跳过
        db_id = vis_entry.get('db_id')
        sql_part = vis_entry.get('vis_query', {}).get('data_part',
        {}).get('sql_part')
        vis_obj = vis_entry.get('vis_obj')
        if not db_id or not sql_part or not vis_obj:
            print(f"警告: ID '{original_id}' 缺少关键信息, 跳过.")
            error_count += 1
            continue

        # 4. 提取所有相关的表名
        all_table_names = extract_all_table_names(sql_part)
        if not all_table_names: # 如果无法提取表名
            print(f"警告: ID '{original_id}' 的 SQL 无法提取表名, 跳
            过。SQL: {sql_part}")
            error_count += 1
            continue

        # 5. 确定主表名和主数据文件路径 (用于顶层 data_file)
        primary_table_name = all_table_names[0] # 使用列表中的第一个
        作为主表

        # 使用 os.path.join 构建跨平台的路径
        # base_data_dir 是 VisEval 数据集根目录的相对路径
        # .replace('\\', '/') 确保路径分隔符是 '/'
```

```

primary_data_file_path = os.path.join(base_data_dir,
'databases', db_id, f"{primary_table_name}.csv").replace('\\', '/')

# 6. 构建包含所有涉及的数据文件路径的列表
involved_data_files = [
    os.path.join(base_data_dir, 'databases', db_id, f"
{name}.csv").replace('\\', '/')
    for name in all_table_names # 遍历所有提取到的表名
]

# 7. 遍历当前条目的每一个 nl_query, 为其创建 BA-bench 样本
for nl_idx, nl_query in enumerate(nl_queries):
    # 8. 创建 BA-bench 样本字典
    ba_bench_sample = {
        'id': f"VisEval_{original_id}_{nl_idx}", # 生成唯一
ID
        'question': nl_query, # 当前的
nl_query
        'data_file': primary_data_file_path, # 主数据文件
路径 (str)
        'doc_file': "None",
        'answer': json.dumps(vis_obj), # vis_obj
序列化为 JSON 字符串
        'data_domain': "General Visualization",
        'analysis_type': "Chart problems",
        'origin_from': ['VisEval', original_id],
        'additional_information': { # 填充附加
信息
            'hardness': vis_entry.get('hardness'),
            'chart': vis_entry.get('chart'),
            'vis_query': vis_entry.get('vis_query'),
            'sql_part': sql_part,
            'involved_data_files': involved_data_files, #
**包含所有文件路径的列表**
            'irrelevant_tables':
vis_entry.get('irrelevant_tables', []),
            'query_meta': vis_entry.get('query_meta', [])
        }
    }

# 9. 将生成的样本添加到结果列表中
ba_bench_data.append(ba_bench_sample)
processed_count += 1 # 增加处理计数

```



```

# 10. 确保输出目录存在, 如果不存在则创建
output_dir = os.path.dirname(output_json_path) # 获取输出文件所
在的目录
if output_dir and not os.path.exists(output_dir): # 如果目录非空
且不存在
    try:
        os.makedirs(output_dir) # 创建目录 (包括任何必要的父目
录)

        print(f"创建目录: {output_dir}")
    except OSError as e: # 处理创建目录时可能发生的错误
        print(f"错误: 创建目录失败 {output_dir}: {e}")
        return # 创建失败则退出

# 11. 将结果列表写入输出 JSON 文件
try:
    with open(output_json_path, 'w', encoding='utf-8') as f:
        # indent=4 使 JSON 文件格式化, 易于阅读
        # ensure_ascii=False 允许非 ASCII 字符 (如中文) 直接写入,
而不是转义
        json.dump(ba_bench_data, f, indent=4,
ensure_ascii=False)
        # 打印成功信息和统计数据
        print(f"转换完成! 成功处理 {processed_count} 个查询。")
        if error_count > 0:
            print(f"跳过 {error_count} 个有问题的条目/查询。")
        print(f"结果已保存到: {output_json_path}")
except IOError as e: # 处理文件写入错误
    print(f"错误: 无法写入输出文件 {output_json_path}: {e}")

```

- **核心逻辑:** 该函数是转换过程的主体。它按顺序执行上述步骤: 加载数据 → 遍历条目 → 遍历查询 → 提取信息 → 构建路径 → 创建样本 → 保存结果
- **路径构建:** 使用 `os.path.join` 来确保生成的路径在不同操作系统 (Windows, Linux, macOS) 上都是有效的。使用 `.replace('\\', '/')` 将 Windows 的反斜杠 `\` 统一转换成正斜杠 `/`, 这通常在跨平台应用和 Web 相关的路径中更常用。路径是相对于执行脚本的工作区根目录构建的
- **多文件处理:** 如策略中所述, 它提取所有表名, 用第一个构建 `data_file`, 然后将所有对应的文件路径列表存储在 `additional_information['involved_data_files']` 中
- **错误处理与健壮性:** 代码包含对文件未找到、JSON 解析错误、关键信息缺失、无法提取表名、无法创建目录和无法写入文件等情况的基本处理和警告/错误提示
- **JSON 输出:** 使用 `json.dump` 将包含所有样本的 Python 列表写入目标文件, `indent=4` 参数使输出的 JSON 文件具有良好的缩进格式, 便于人工阅读; `ensure_ascii=False` 对于处理可能存在于数据中的非英文字符重要

### 5.3. 脚本执行与路径配置

脚本末尾的“使用示例”部分负责配置输入输出路径并调用转换函数：

PYTHON

```
# --- 使用示例 ---
# 假设 VisEval 数据集在 VisEval/visEval_dataset
# 假设此脚本在 TransVis

# 输入文件（相对于工作区根目录）
input_file = 'VisEval/visEval_dataset/visEval.json'
# VisEval 数据集基础目录（用于构建内部数据库文件路径）
viseval_base_dir = 'VisEval/visEval_dataset'
# 输出目录（脚本所在的 TransVis 目录）
output_dir = 'TransVis'
# 输出文件名
output_filename = 'ba_bench_viseval.json'
# 组合最终输出路径
output_file = os.path.join(output_dir,
output_filename).replace('\\', '/')

# 执行主转换
print(f"开始转换: {input_file} -> {output_file}")
convert_viseval_to_ba_bench(input_file, output_file,
viseval_base_dir)

# （注释掉的部分是处理 single/multiple 文件的示例代码）

print("\n所有处理结束。")
```

- **路径假设:** 这部分代码假设 `VisEval/visEval_dataset` 目录和 `TransVis` 目录都位于运行 Python 脚本时的工作区根目录下
- **配置:** 它设置了输入 JSON 文件 (`visEval.json`) 的路径、VisEval 数据集的基础目录 (`viseval_base_dir`，用于查找 `databases/` 下的文件)、输出目录 (`TransVis`) 和输出文件名
- **执行:** 最后调用 `convert_viseval_to_ba_bench` 函数，传入这些配置好的路径来启动转换过程

## 6. 数据格式建议修改

由于VisEval中存在多个自然语言问题对应同一数据集及数据处理方式，同时一问题可能对应多个数据集，而结果为对于数据图表的描述，建议修改现有数据格式如下：

- question字段: str→list[str]
- data\_file字段: str→list[str]
- answer字段: str→json

上述修改不仅可用于VisEval数据集, 对于同数据问题多提问方式、同数据问题多数据集、及格式化输出控制的其他数据集均可进行处理